

# 大数据科学及数据科学与模型产

从混乱到洞见，从实验到生产

## 第九周：数据产品交付 (3)

### Kaggle 实战方法论与大作业部署

#### [解释]

本节课是数据产品交付系列的第三讲，也是最后一讲。前两讲分别介绍了分析报告写作（9-1）和数据看板设计（9-2），本课将进入 Kaggle 竞赛实战和期末大作业部署。

核心内容包括：竞赛的标准工作流、模型融合策略、以及期末项目的评分标准和交付要求。

## 课程定位：从理论学习到实战交付

- W1-W7：技术能力建设
- W8：模型治理与 DataOps
- W9：LLM 工程实践与数据产品交付（本周）
  - 9-1：数据叙事与分析报告写作
  - 9-2：数据看板交付方法论
  - 9-3：Kaggle 实战方法论与大作业部署（本节）
- W10-W11：大作业展示与答辩

### 本讲定位

理论学习 → 实战转化

把前 8 周的技术能力和本周学到的叙事与交付能力，转化为可交付的竞赛成果和项目文档。

### [解释]

本页展示了这节课在整个课程体系中的位置。

前 8 周建立了技术能力，本周前两讲分别解决了“怎么写报告”和“怎么做看板”的问题。

本讲（9-3）是将所有这些能力转化为实际竞赛成果和项目文档的实战环节。

## 本课概览：学完本课后，你能做到什么？

1. **[应用层]** 贯穿 Kaggle 竞赛生命周期，**执行**从基线构建、特征提取、验证集切分到线上提交的系统化流水线，规避无序的盲目调参。
2. **[分析层]** 拆解不同模型融合架构（如 Bagging、Stacking）的底层机制，**界定**多模型集成策略在防过拟合与提升预测界限上的有效收益。
3. **[评价层]** 运用消融实验（Ablation Study）范式，对特定特征或组件的有效性进行**定量测度**，建立工程维度的代码可复现置信度基准。
4. **[创造层]** 整合贯穿整学期的数据分析与模型资产，**构建**符合工业运行闭环标准与严格隔离规范的期末大作业综合交付包。

### **[解释]**

本课有四个学习目标，对应 Bloom 认知层级从低到高：

应用层：能执行完整的 Kaggle 六阶段工作流。

分析层：能分析不同模型融合策略的原理和适用场景。

评价层：能用消融实验评价特征的有效性。

创造层：能组装符合交付标准的期末项目工程包。

# 一、Kaggle 竞赛的系统化工作流

不靠灵感靠流程，把竞赛变成工程项目

## 【解释】

第一章介绍 Kaggle 竞赛的标准工作流。

很多初学者参加竞赛时没有计划，随意调参、盲目提交。

本章将竞赛过程拆解为六个有明确输入输出的阶段，帮助建立系统化的工作习惯。

## 引言案例：Kaggle 典型 Shake-up 案例

**真实案例：**在 [Mercedes-Benz Greener Manufacturing](#) 竞赛中，某团队凭借复杂的堆叠模型和大量的特征堆砌，在 Public Leaderboard (公开榜单) 上名列前茅。然而，当比赛结束，官方切换到用于最终结算的 Private Leaderboard (私有榜单) 时，该团队的排名从前列大幅下滑至 **第 2125 名**。这是一个典型的过拟合导致 Shake-up 的反面案例。

### 原因分析（验证缺失与工作流问题）：

该团队陷入了“排行榜驱动开发”的反模式。他们没有建立严密的本地交叉验证（Local CV）架构，而是盲目凭借提交 Public LB 的得分来试探特征。结果模型过拟合了公开测试集的特征噪声，泛化能力严重下降。在工业界，这种未经工程验证的模型一旦上线，将直接导致业务指标恶化。

### 【解释】

本页通过一个真实的 Kaggle 竞赛案例说明 Shake-up 现象的严重性。

该团队没有建立本地交叉验证，完全依赖 Public LB 分数来判断模型好坏，导致模型过拟合了公开测试集的噪声，最终排名从前列跌至第 2125 名。

这个反面案例引出了本章的核心主题：系统化的工作流和可靠的本地验证是竞赛成功的底线。

# 六阶段竞赛 workflow

| 阶段         | 核心任务                      | 对应课程内容            | 常见错误                |
|------------|---------------------------|-------------------|---------------------|
| ① 问题理解     | 精读竞赛说明、理解评估指标、分析数据格式      | W1 大数据基础          | 不看评估指标就开始写代码        |
| ② EDA      | 数据探索、缺失值分析、分布可视化          | W4 探索性分析          | 跳过 EDA 直接建模         |
| ③ Baseline | 用最简单的方法建立基准分数             | W6 模型工程化          | 第一版就用复杂模型           |
| ④ 特征工程     | 逐个添加特征，用消融实验验证            | W4 特征工程 + W8 消融实验 | 一次性加入大量未验证特征        |
| ⑤ 模型融合     | 多模型组合提升稳定性                | 本讲                | 融合过多导致过拟合 Public LB |
| ⑥ 后处理      | 提交策略、Public/Private LB 管理 | 本讲                | 只优化 Public LB       |

**核心原则：**先建 Baseline，再逐步迭代。每一步都要有量化对比（ $\Delta$ Metric），不要做无法衡量效果的改动。

**[解释]**

本页表格将 Kaggle 竞赛拆解为六个阶段，每个阶段对应前几周学过的课程内容。  
核心原则是“先建 Baseline，再逐步迭代”：在使用复杂模型之前，先用最简单的方法跑出一个基准分数。之后每次改进都要和这个基准对比，计算提升幅度（ $\Delta$ Metric），确保每一步改动都有量化依据。

# Baseline 的工程价值：为什么越简单越好？

## 推荐系统的 Baseline 示例：全站热门商品推荐

```
top12 = df.groupby('article_id')['t_dat'] \
    .count().nlargest(12).index.tolist()

submission['prediction'] = ' '.join(
    [str(x) for x in top12]
)
```

- 不需要特征工程
- 不需要训练模型
- 几行代码，几分钟出结果

## Baseline 的三个作用

1. 建立参考系：后续改进的量化基准
2. 验证管道：确认数据加载、格式转换、提交流程都是通的
3. 暴露问题：如果复杂模型打不过 Baseline，说明系统存在根本问题

**经验法则：**在 Kaggle 上，能打败“全站热门”这个 Baseline 的推荐系统，已经超过了约 30% 的参赛者。

### [解释]

本页说明了 Baseline 的三个工程价值：建立参考系、验证管道、暴露问题。

Baseline 越简单越好，因为它的核心价值不是“拿高分”，而是“确认整个流水线是通的”。

右侧的代码示例展示了一个最简单的推荐系统 Baseline：给每个用户推荐全站最热门的 12 件商品，不需要任何特征工程。

## 工业案例：Netflix Prize 的工程权衡

**映射知识点：**过度复杂模型在工业落地时的 ROI（投资回报率）考量。

著名的 Netflix Prize 百万美元大奖，其冠军方案是由 107 个算法组成的超级集成模型（BellKor's Pragmatic Chaos）。

然而，由于该系统工程复杂度极高，Netflix 最终判定其部署维护成本远超算法带来的边际收益，因此**从未真正在生产环境中全量部署该获奖模型**，而是选择了性能稍逊但架构简单的 Baseline 级变体。

这印证了在工业界，Baseline 不仅是起跑线，很多时候更是系统架构权衡后最终上线的首选。

### **[解释]**

Baseline 的价值不仅是提供一个分数参考，更重要的是验证整个流水线的连通性。

通过先跑通一个最简单的模型，可以确认数据加载、格式转换、提交流程等环节都没有问题。如果连 Baseline 都跑不通，说明流水线本身有 bug，此时追求复杂模型毫无意义。

Netflix Prize 案例进一步说明：在工业界，Baseline 级别的简单模型往往因为维护成本低，反而是最终上线的首选。



## 二、模型融合的工程策略

模型集成：降低单点失效风险的标准手段

### 【解释】

第二章从“单模型调优”转向“多模型组合”。

第一章建立了竞赛的系统化工作流，其中第五阶段“模型融合”是当单模型性能达到瓶颈后的标准突破方法。

本章介绍三种主流融合策略（简单平均、Bagging、Stacking），并通过工业案例说明异构融合的工程价值。

# 三种主流融合策略概览

| 策略             | 原理                              | 实现复杂度 | 典型提升 |
|----------------|---------------------------------|-------|------|
| 简单 / 加权平均      | 对多个模型的预测结果直接取平均或赋予不同权重          | 低     | 1-3% |
| Bagging（并行融合）  | 多个同类模型在不同的数据或特征子集上训练，降低方差       | 中     | 2-5% |
| Stacking（分层融合） | 用一个元模型（Meta-Learner）学习如何组合多个基模型 | 高     | 3-8% |

## 最基础的融合方式：简单平均代码示例

在没有精力构建复杂工程时，直接平均来自异构模型（如基于树的 XGBoost 与基于梯度的 CatBoost）的预测结果，是最稳妥的保底策略：

```
# 极简且高度稳定的加权融合
pred_final = (0.4 * pred_lgb) + (0.4 * pred_xgb) + (0.2 * pred_cat)
```

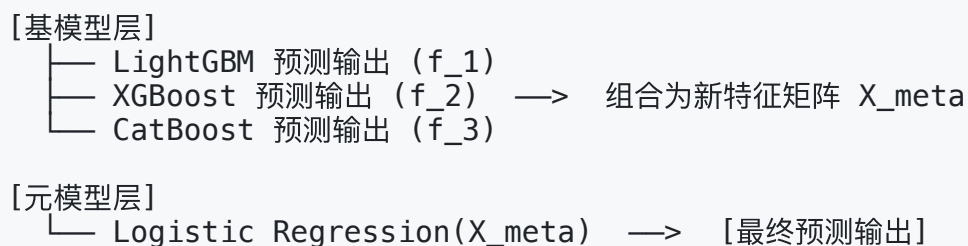
### [解释]

本页介绍三种模型融合策略，从简单到复杂。模型融合有效的前提是参与融合的模型之间要有差异性（即“异构”），例如使用不同的算法或不同的特征集。  
如果多个模型高度相似，融合后效果提升有限。表格中的“简单平均”实现最容易，适合快速尝试；Stacking 最强大但也最容易过拟合，需要谨慎使用。

# 高阶融合：Stacking 架构与融合陷阱

## Stacking 分层计算图

在 Stacking 架构中，第一层（基模型）输出的不是最终结果，而是高阶抽象特征，供第二层（元模型）进行非线性组合。



### 融合陷阱（Over-ensembling）：

过度融合（如堆叠 20+ 个同质化的树模型）不仅会显著拖慢推理速度，更会严重过拟合 Public LB。工业界经验：挑选 3-5 个具有高差异性（如分别偏重不同特征群、或分属不同算法族）的模型进行融合，往往能取得最优的稳定性提升。

#### [解释]

本页介绍了 Stacking 的两层架构：第一层的基模型输出预测结果，作为第二层元模型的输入特征。  
关键注意点：模型数量不是越多越好，3-5 个具有高差异性的模型通常能取得最优效果。  
堆叠太多同质化模型会导致过拟合（Over-ensembling），这是融合阶段最常见的错误。

## 工业案例：蚂蚁集团的风控异构 Stacking 架构

**映射知识点：**异构模型融合在防御系统级单点失效中的作用。

在支付级反欺诈系统中，为了防止单一算法（如某个深度神经网络）在面对黑产的对抗样本攻击时整体失效，工业级系统通常会集成规则引擎（Rule-based）、树模型（XGBoost）以及图神经网络（GNN）形成复合架构。

这种异构集成的核心目的并非为了在离线测试中多拿 0.01 的 AUC 分数，而是为了构建高系统级容错率，确保当某一类特征或算法失效时，整体防线仍然有效。

### [解释]

三种融合策略从简单到复杂：

简单平均：最容易实现，适合快速尝试。关键是参与融合的模型要有差异性（用不同算法或不同特征集训练）。

Bagging：Random Forest 就是 Bagging 的典型代表。通过在不同数据子集上训练多个模型来降低方差。

Stacking：最强大但也最容易过拟合。用一个简单模型（如 Logistic Regression）来学习如何组合基模型的预测。注意：Stacking 的元模型训练必须用交叉验证的 out-of-fold 预测，否则会严重过拟合。

## Public vs. Private Leaderboard：理解评分机制

### LB 分裂机制（因竞赛而异）

- Kaggle 将测试集严格拆分为两部分，分别用于计算 Public LB (公开榜单) 和 Private LB (私有榜单) 的得分。
- 拆分比例由竞赛主办方设定（不是固定的），常见的切分比例有 30/70、50/50 等。
- 在标准的 Featured 竞赛进行期间，**Private LB 是完全不可见的**，直到竞赛结束那一刻才会根据 Private LB 重新结算最终排名（即发生 Shake-up 现象）。
- **注：**由于本课程大作业使用的是**已结束的历史竞赛**，大家通过 Late Submission 提交时，Public 和 Private 分数会同时显示。

#### [解释]

本页解释 Kaggle 的 Public/Private 双榜单机制。

关键要理解：比赛期间看到的 Public LB 分数只反映测试集中一部分数据的表现，不能代表最终成绩。如果过度针对 Public LB 优化，模型可能只是拟合了这部分数据的噪声，在 Private LB 上反而表现很差（即 Shake-up 现象）。

注意：由于本课程大作业使用的是已结束的历史竞赛，两个分数会同时显示。

# 本地验证 (Local CV): 真实的工程护城河

## 为什么即使能看到线上榜单，本地验证仍然必须存在？

- 过度依赖在线评测平台会陷入“提交限制”的瓶颈。每次提交都需要完整的 I/O 与网络开销。
- 本地 CV 的反馈循环极快，更适合在特征工程阶段进行高频次的消融迭代。

## 本地验证的核心工程价值

1. 彻底解绑：本地 CV 几分钟出结果，摆脱平台网络与每日提交次数的物理限制。
2. 细粒度诊断：本地可以完整提取每折（Fold）的分数分布，精准定位数据泄漏或分布漂移。
3. 闭环可复现：本地实验记录完整保留，期末答辩时可直接提供详实的消融实验日志。
4. 架构仿真：利用 W8 教授的时间切分法做本地评估，比随机切分更逼近真实的工业预测场景。

**防呆法则：**无论竞赛平台是否实时暴露 Private LB，建立可靠的本地验证方案都是底线。本地 CV 的价值不仅在于预测最终成绩，更在于为你提供一个透明、可控、高频的实验沙盒。

### [解释]

本页说明为什么即使能直接看到线上分数，仍然需要建立本地交叉验证（Local CV）。本地验证有四个核心价值：速度快（几分钟出结果，不受平台提交限制）、诊断精细（可以看到每折的分数分布）、完整记录（答辩时可提供实验日志）、更贴近实际（用时间切分法比随机切分更可靠）。

## 阶段检验 (Review Quiz)：决策判断

### 场景：

你正在参加一个竞争激烈的 Kaggle 比赛。你昨天构建的 Baseline 模型在本地交叉验证 (Local CV) 得分是 `0.85`，Public LB 提交得分也是 `0.85`。

今天你花了一整天加入了一个极其复杂的“深度时序交叉特征”。重新验证后：

- 你的 Local CV 得分下降到了 `0.82`。
- 但是你把它提交上去，Public LB 的得分上升到了 `0.89`。

### 关键问题：

面对这种“本地下降，线上上升”的倒挂现象，作为数据科学家，你该不该把这个“交叉特征”合并到模型主干代码中？

### 【解释】

这页用一个“本地分数下降但线上分数上涨”的极端场景，说明应该信任本地验证而不是线上榜单。

正确答案是“不合并”：本地 CV 下降说明该特征在全量数据上是有害的，线上分数上涨大概率是偶然过拟合了 Public 测试集中的噪声。

这个决策原则在工业界同样适用。

## 阶段检验：工程决策正解

### ✗ 绝对不能合并。

本地严密的 CV 出现下降，说明该特征在全量数据分布上是失效甚至有害的。Public LB 出现上升，大概率是因为该特征偶然过拟合了 Public 测试集中那 30% 的分布噪声。

如果在真实的工业界，这相当于为了短期通过特定测试，而在系统中引入了将在全量部署时暴露的缺陷。

**防呆法则：**永远相信你严密构建的本地验证流，而不是短视的黑盒榜单。

#### [解释]

虽然本课程大作业使用的是已结束的竞赛（可以直接看到 Private LB），但建立本地验证机制仍然是必要的。原因有两个：一是本地验证的迭代速度远快于在线提交，适合高频的特征工程实验；二是本地验证可以提供详细的每折分数分布，帮助定位数据泄漏或分布漂移问题。

特别提醒：对于推荐系统等时序业务，必须使用时间切分法做本地验证。



## 高阶认知：工业级 DataOps 的工程陷阱 (上)

### 🚩 陷阱 1：验证集时序泄露 (Time-Leakage)

在推荐、销量预测等强时效性业务中，**绝对禁止使用随机交叉验证 (Random K-Fold)**。

- **病理**：随机抽样会让模型“偷看”到未来的数据分布，导致评测指标极高，但上线即崩溃。
- **药方**：必须严格遵循**按时间滑动窗口切分 (Time-based Split)**。永远只用“过去的完整闭环”去预测“未来的完整窗口”。

### 🚩 陷阱 2：拒绝特征的打包提交

很多新手习惯花一周时间堆砌 10 个特征，然后一把跑出结果。

- **病理**：如果这 10 个特征里有 3 个是正向收益，7 个是负向收益，最终的分数微涨掩盖了多数特征的负面影响。
- **药方**：必须执行严格的**单一变量消融法则 (Single-Variable Ablation)**。一次只动一个变量。无法精确定位收益来源的工程实验，即使涨分了也是彻底失败的实验。

#### [解释]

本页介绍两个常见的工程陷阱。

陷阱 1（时序泄露）：在推荐、销量预测等场景中，如果使用随机交叉验证，模型会“偷看”到未来的数据，导致本地评测指标虚高，上线后性能急剧下降。必须使用按时间滑动窗口切分的方式。

陷阱 2（打包提交）：一次加入多个特征后无法判断哪些有效、哪些有害，必须一次只改一个变量（消融实验原则）。

## 高阶认知：工业级 DataOps 的工程陷阱 (下)

### 🚩 陷阱 3：树模型的“特征惰性”

不能假设把所有特征丢给模型，它就能自动选出最好的。

- **病理**：当存在一个很强的信号（如“用户总购买数”）时，模型会过度依赖该特征产生**路径依赖**，导致如“价格偏离度”等精细特征的贡献被遮蔽（SHAP $\approx$ 0）。
- **药方**：人工进行**等频分箱 (Quantile Binning)**，将连续分布强行离散化，强迫决策树在休眠的微观特征上进行信息增益计算。

### 🚩 陷阱 4：召回扩容导致的精度下降

很多团队发现 Recall 遇到了天花板，便盲目扩大召回候选集的数量（如从 K=150 扩大到 K=500）。

- **病理**：在排序器 (Ranker) 微观分辨力没有质变的前提下，盲目扩大 K 值相当于在候选集中引入了大量低质量样本。
- **药方**：低质量的候选项会把原本排在前列的正确答案挤出 Top-K。工业共识：排序模型的最终精度，永远与候选池的信噪比紧密绑定。

#### [解释]

本页介绍另外两个高阶陷阱。

陷阱 3（特征惰性）：当特征集中存在一个很强的信号时，树模型会过度依赖它，导致其他精细特征的贡献被遮蔽（SHAP $\approx$ 0）。解决方法是对强特征做等频分箱，强迫模型关注其他特征。

陷阱 4（召回扩容）：盲目扩大召回候选集的数量，会引入大量低质量样本，反而降低排序精度。

# 三、大作业部署：评分规则与交付要求

明确目标，制定交付计划

## 【解释】

第三章从“竞赛方法论”转向“工程交付”。

前两章分别解决了“怎么系统化地做竞赛”和“怎么用多模型提升性能”的问题。

本章要解决的是“做出来之后怎么交付”——包括评分规则、文档要求和代码可复现性标准。

这部分内容直接关系期末成绩，建议仔细阅读每一项要求。

# 期末考核评分体系

| 评价环节        | 权重  | 考核内容                                 |
|-------------|-----|--------------------------------------|
| 平时成绩        | 30% | 考勤、课堂互动、单元测验或课后编程作业                  |
| 期中 EDA 报告   | 20% | 数据集特征分析、清洗策略、特征工程思路、Baseline 结果及改进方向 |
| Kaggle 竞赛排名 | 15% | Private Leaderboard 最终排名赋分           |
| 项目文档报告      | 20% | 特征工程策略选择依据、失败尝试的分析、数据分析流水线架构设计       |
| 答辩与个人贡献     | 15% | 小组路演 + 教师提问，根据 Git 记录与答辩表现区分个人成绩     |

**关键提醒：**Kaggle 排名只占 15%，但项目文档占 20%、答辩占 15%。排名不是唯一决定因素——一个排名中等但文档清晰、失败分析深入的团队，总分可能高于排名靠前但文档敷衍的团队。

**[解释]**

注意评分权重的分配：Kaggle 排名只占 15%，而项目文档（20%）和答辩（15%）合计占 35%。这意味着分数的主要来源不是“跑得多高”，而是“能不能说清楚你为什么这么做”。文档中需要包含决策过程、失败分析和架构说明。另外，Git 提交记录会作为答辩时区分个人贡献的依据。

# 项目文档的核心要求

项目文档不是"实验日志", 而是一份**工程决策报告**。评分重点不是"你试了什么", 而是"你为什么这么决定"。

## 必须包含的内容

1. **数据分析**: EDA 发现的关键特征和数据质量问题
2. **特征工程策略**: 每个特征的设计动机和消融实验结果
3. **失败分析**: 尝试过但效果不好的方法, 以及原因分析
4. **流水线架构**: 数据处理和模型训练的完整流程图

## 评分标准

- **A 档**: 决策有数据支撑, 失败分析深入, 架构设计清晰
- **B 档**: 内容完整但分析深度不足
- **C 档**: 仅罗列实验结果, 缺乏分析和决策依据
- **D 档**: 内容不完整或明显抄袭

**关于"失败分析"**: 一个有深度的失败分析比一个没有分析的成功更有学术价值。例如: "我们尝试加入用户年龄特征, MAP 反而下降了 0.003。分析原因: 年龄数据缺失率 40%, 填充策略引入了噪声。"——这比"我们加了年龄特征, 效果不好就删了"有价值得多。(详见下一页)

### [解释]

本页展示了消融实验结果和失败分析的标准呈现方式。

左侧的  $\Delta$ Metric 表格是必须的: 每加入一个特征, 记录指标变化和是否保留的判定。

右侧的"失败分析五步法"来自 Kaggle 社区的规范: 按动机→方法→结果→归因→教训的结构记录每个失败的尝试。这种写法比简单的"加了没用就删了"有价值得多。

# 大作业进阶：消融实验与失败分析的呈现范式

## 消融实验结果呈现

标准做法： $\Delta$ Metric 表格 + 瀑布图

| 特征       | +特征后   | $\Delta$ Metric | 判定   |
|----------|--------|-----------------|------|
| Baseline | 0.0234 | -               | -    |
| 购买频率     | 0.0261 | +0.0027         | ✅ 保留 |
| 用户年龄     | 0.0231 | -0.003          | ❌ 放弃 |

- **表格**：给出精确数值，便于复现核查。
- **瀑布图**：视觉直观，正贡献用绿色，负贡献用红色。

## 失败分析写作五步法

Kaggle 社区的 "blameless sharing" 规范：

1. **动机 (Motivation)**：为什么想做这个尝试？
2. **方法 (Method)**：具体是怎么实现的？
3. **结果 (Result)**：量化指标变化（如 MAP 下降 0.003）。
4. **归因 (Attribution)**：数据层面为什么没用？（如分布偏斜）。
5. **教训 (Lesson)**：对后续实验有什么指导意义？

### [解释]

上一页展示了消融实验和失败分析的两种标准呈现方式。  
消融实验用  $\Delta$ Metric 表格记录每个特征的增量贡献，配合瀑布图直观显示正负贡献。  
失败分析按五步法（动机→方法→结果→归因→教训）记录，目的是积累团队经验，避免重复踩坑。

## 代码可复现性要求

项目文档必须附带以下信息，确保评委或团队成员能完整复现结果：

| 要素                            | 说明                                          |
|-------------------------------|---------------------------------------------|
| <code>requirements.txt</code> | 精确到版本号（ <code>lightgbm==4.1.0</code> ）      |
| 随机种子                          | 所有涉及随机性的步骤都固定 seed                          |
| 数据版本                          | 明确使用的数据集版本或下载日期                             |
| 运行说明                          | 一条命令跑通全流程（ <code>bash run_train.sh</code> ） |
| 硬件环境                          | 训练用了多少内存 / GPU / 耗时多久                       |

**工程标准：**在实际生产中，缺乏可复现性的模型结论将无法进行工程转化。大作业交付包中若缺少环境依赖与随机种子声明，会大幅降低评委对实验结果的置信度评价。

### [解释]

本页列出了代码可复现性的五个要素。最关键的三个是：`requirements.txt` 必须精确到版本号（如 `lightgbm==4.1.0`），所有随机步骤必须固定 seed，以及必须提供一条命令跑通全流程的运行说明。如果缺少这些信息，评委无法复现你的实验结果，会大幅降低对结论的可信度评价。

## 工业案例：金融量化领域的“回测陷阱”

**映射知识点：** 随机种子固定与环境依赖（requirements）锁定的刚性要求。

在某顶级量化基金中，某策略团队在本地的回测收益曲线表现良好，但上线生产环境后直接导致**显著亏损**。事后复盘发现，原因仅仅是因为本地与线上的 Pandas 版本不同，导致 `groupby` 操作后的默认排序逻辑发生了微小偏差，进而使得时序特征全部错位。

在现代工业级的 DataOps 中，任何无法通过容器一键复现（Reproducible）的代码提交，都会被 CI/CD 引擎**直接拦截**，剥夺上线资格。

### [解释]

项目文档的评分重点是“决策过程”而不是“最终结果”。

一个排名中等但决策过程清晰、失败分析深入的报告，评分会高于排名靠前但只写了“用了 LightGBM，调了参数”的报告。

特别强调“失败分析”的价值：在工业界，记录和分析失败尝试是团队知识积累的核心手段。一个好的工程师不仅知道什么有效，更知道什么无效以及为什么。



## 本课总结：三个核心要点

1. **系统化工作流**：Kaggle 竞赛是工程项目，不是碰运气。六阶段流程（问题理解 → EDA → Baseline → 特征工程 → 模型融合 → 后处理）确保每一步都有量化依据。
2. **模型融合的关键是差异性**：3-5 个差异化模型的融合效果通常优于单一模型。无论竞赛是否实时显示 Private LB，建立可靠的本地验证方案都是必须的。
3. **文档和答辩的权重高于排名**：Kaggle 排名占 15%，但文档 + 答辩占 35%。用 9-1 学到的数据叙事方法论写报告，用 9-2 学到的看板方法论做可视化交付。

**核心观点**：一个好的数据科学项目，不仅要“做出来”，更要“说清楚”——说清楚你为什么这么做、试过什么、放弃了什么、最终选择了什么。这就是从“写代码”到“做工程”的跨越。

### [解释]

本课总结了三个核心要点：一是竞赛要按六阶段工作流系统化推进；二是模型融合要注重差异性并建立可靠的本地验证；三是文档和答辩的评分权重高于 Kaggle 排名。

整合 9-1 的数据叙事方法论和 9-2 的看板方法论，完成从“写代码”到“做工程”的转变。

# 第九周：大作业交付与实战·课后作业

## 实操题🔧：本地验证与特征消融实验

**任务：**在期末大作业（Kaggle H&M 推荐系统）的代码基础上，跑通本地验证流程，完成以下两个小实验并把结果写进报告里：

- 1. 防穿越的本地验证：**代码里必须按时间顺序去切分验证集（也就是让验证集的时间晚于训练集）。跑一下你最简单的 Baseline 模型，记录它在本地（Local CV）和线上（Public LB）的分数分别是多少。另外，用自己的话解释一下：做这种推荐系统的项目，为什么坚决不能用随机打乱的交叉验证？
- 2. 做个标准的消融实验：**选出你最近刚写好的 3 个新特征。记住“一次只加一个变量”的原则，把它们分别加进模型里跑一下本地验证。最后画一个表格（列出：特征名字、加入后的分数、分数的变化量、最后决定保留还是删掉），展示你的实验过程。

## 分析题📊：分数倒挂与最终的决策

**场景：**距离大作业截止只剩三天了，你们团队遇到了两个棘手的情况：

- 现象 A：你们用 LightGBM 和 XGBoost 分别跑出了两个模型，线上分数都是 0.023。有组员说：“要不咱们直接把这两个模型的结果按 1:1 平均一下，分数肯定还能涨！”
- 现象 B：有人在论坛里抄来了一个“开源魔法特征”，加进去之后，线上分数（Public LB）竟然从 0.023 暴涨到了 0.028。但奇怪的是，你们本地严密测试的分数（Local CV）反而从 0.021 跌到了 0.014。

**思考：**请结合课上讲过的“模型融合”和“双榜单机制”，分别评价一下现象 A 的做法靠不靠谱，以及现象 B 背后可能藏着什么坑。针对现象 B 这种巨大的涨分诱惑，为了防止最后出成绩时遭遇大翻车（Shake-up），如果你是组长，你会同意把这个“魔法特征”合进最终的代码里吗？请给出你的理由。